



Universidad Veracruzana

Universidad Veracruzana

Facultad de Estadística e Informática
Licenciatura en Ingeniería de Software

Diseño de Software

Diseño arquitectónico

ASR, drivers y primera iteración de Bastion

Presenta: Angel Gabriel Aguilar Hernández
José Eduardo Prior Hernández

Docente: Polo Estrella Ana Luz

23 de septiembre de 2026

Índice

1. Método y estructura del documento	1
1.1. Documentos de entrada	1
1.2. Orden del documento	1
1.3. Cómo se enuncian los requisitos y los drivers	1
1.4. Los pasos del método y sus entradas	2
1.5. Estructura de una iteración	2
1.6. Convenciones	3
2. ASR	3
2.1. Cómo se identifican	3
2.2. Cómo se enuncia un ASR	4
2.3. ASR encontrados	5
2.4. Por qué cada uno es ASR	6
2.5. Requisitos y decisiones que no son ASR	8
2.6. Priorización	9
2.7. Resultado	11
3. Drivers	11
3.1. Cómo se convierte un ASR en driver	11
3.2. Los cinco drivers	12
3.3. Prioridad de los drivers	13
3.4. Lo que queda fuera de estos cinco drivers	14
4. Iteración 1 — Partida en línea íntegra	14
4.1. Alcance de la iteración	14
4.2. Paso 1. Revisión de entradas	15
4.3. Paso 2. Objetivo de la iteración y selección de entradas	18
4.4. Paso 3. Elemento a descomponer	19
4.5. Paso 4. Conceptos de diseño	20
4.6. Fichas de decisión	24

1. Método y estructura del documento

Este documento aplica el método ADD a Bastion. Primero identifica los requisitos arquitectónicamente significativos, después los convierte en drivers y, por último, desarrolla la primera iteración de diseño: revisa las entradas que corresponden a los drivers de esa iteración, fija un objetivo, elige qué elemento del sistema descompone, selecciona los conceptos de diseño con los que lo hace y registra cada decisión con las alternativas que se compararon y con lo que la decisión cuesta.

1.1 Documentos de entrada

Documento	Qué aporta
Análisis de contexto, escenarios de calidad y drivers	Objetivos de diseño, stakeholders (STK), concerns (CRN), dependencias (DEP), restricciones (CON) y requisitos impuestos (REQ), tensiones (TEN), preguntas abiertas (Q), características de calidad (QA) y escenarios (ESC) con su Utility Tree. Sus ASR y sus drivers son un antecedente: este documento los vuelve a identificar en las secciones 2 y 3.
Casos de uso para Bastion, versión 1.0	Los requisitos funcionales primarios (CU), con sus flujos, precondiciones, reglas de negocio (RN), mensajes del protocolo y entidades de datos. Es la fuente de lo que el sistema tiene que hacer, frente a los drivers, que dicen qué lo vuelve difícil.
Bass, Clements y Kazman, <i>Software Architecture in Practice</i> , cuarta edición	El método. De su capítulo 19 sale la definición de ASR y la prueba con la que se identifican; de su capítulo 20, los pasos de ADD, los tipos de concepto de diseño y la forma de compararlos por sus ventajas y desventajas antes de elegir.

Los identificadores se conservan tal como aparecen en esos documentos, de modo que cualquier afirmación de este texto se pueda rastrear hasta su origen sin repetirlo aquí.

1.2 Orden del documento

Las secciones se construyen en cadena, y cada una depende solo de las anteriores.

Sección	Qué toma y qué entrega
2. ASR	Toma solo los documentos de entrada. Entrega los quince ASR con su procedencia y su prioridad, y las decisiones del equipo (DEC-01 a DEC-03).
3. Drivers	Toma los cinco ASR de mayor prioridad. Entrega un driver por cada uno, más general que su ASR y sin soluciones dentro, con el orden heredado.
4. Iteración 1	Toma los drivers que le corresponden y los ASR que hay detrás. Entrega un objetivo, un elemento descompuesto, los conceptos de diseño elegidos y las fichas de decisión que los justifican.

1.3 Cómo se enuncian los requisitos y los drivers

Una regla atraviesa las tres secciones: un requisito dice **qué** tiene que lograr el sistema y un driver dice **qué problema** plantea ese requisito, pero ninguno de los dos dice **cómo** se resuelve. Las soluciones aparecen por primera vez en el paso 4 de la iteración, y siempre acompañadas de las alternativas con las que se compararon. La justificación de la regla y los enunciados que hubo que corregir están en las subsecciones 2.2 y 3.

La única excepción son las restricciones, que ya son decisiones tomadas por quien tiene autoridad para tomarlas: se enuncian tal como llegan, y lo que se les exige es la justificación de por qué son

arquitectónicamente significativas.

1.4 Los pasos del método y sus entradas

Las entradas del método son cinco: objetivos de diseño, requisitos funcionales primarios, escenarios de atributos de calidad, restricciones y concerns. Los pasos que cubre este documento son los siguientes.

Paso	Nombre	Qué produce en este documento
Paso 1	Revisión de entradas	La lista de entradas que tienen algo que ver con los drivers de la iteración, cada una con lo que exige. Se revisan todas; se listan las que aplican.
Paso 2	Establecer el objetivo de la iteración y seleccionar las entradas	Un objetivo enunciado en una frase, que satisface los ASR de los drivers de la iteración y es alcanzable dentro de ella, con la medida que lo comprueba y el incremento que produce. Y la tabla que pesa cada entrada y dice cuáles se atienden ahora y cuáles se difieren, con el motivo.
Paso 3	Elegir uno o más elementos a descomponer	La ficha del único elemento elegido: qué abarca, por qué se eligió, qué falta decidir sobre él y su trazabilidad. La razón nunca es que sea un módulo del sistema, sino que la medida del objetivo depende de decisiones que todavía no se han tomado sobre él.
Paso 4	Elegir los conceptos de diseño que satisfacen las entradas	Los conceptos seleccionados, uno por ficha, con su estilo, qué atiende y por qué se eligió frente a las alternativas, sus efectos y su trazabilidad. Con ellos, el elemento del paso 3 queda partido en elementos concretos (C-...) y aparecen los riesgos que quedan abiertos (R-...).
Cierre	Registro de las decisiones	Una ficha por decisión (ADR-...) con su estado, el contexto que obliga a decidir, las alternativas con sus ventajas y desventajas, los criterios tomados de los ASR, la decisión elegida, lo que se ganó, lo que se pagó, los riesgos y los elementos afectados.

Un driver no se “agota” en una iteración: la iteración lo atiende hasta donde su objetivo alcanza, y lo que queda se nombra al diferirlo. La instanciación completa de los elementos, la definición de sus interfaces y el análisis del avance quedan fuera del alcance de este documento.

1.5 Estructura de una iteración

Todas las iteraciones tienen las mismas subsecciones, en el mismo orden, y terminan en las fichas de decisión.

Subsección	Contenido
Alcance de la iteración	Los drivers que cubre, por qué esos y no otros, qué decide, qué no decide y con qué decisiones previas cuenta.
Paso 1. Revisión de entradas	Primero, los drivers de la iteración con el ASR que hay detrás de cada uno. Después, las cinco entradas del método filtradas por esos drivers, con las decisiones ya tomadas y las preguntas abiertas que las condicionan.
Paso 2. Objetivo de la iteración y selección de entradas	La tabla que pesa las entradas y el objetivo con su medida, su límite y el incremento que produce.
Paso 3. Elemento a descomponer	Una ficha con el elemento elegido: qué abarca, por qué se eligió, qué falta decidir y su trazabilidad.
Paso 4. Conceptos de diseño	Una ficha por concepto seleccionado: estilo, descripción, efectos y trazabilidad. Cierra con los elementos resultantes y los riesgos abiertos.

Subsección	Contenido
Fichas de decisión	Una ficha por decisión, con sus alternativas, sus criterios, lo que se ganó y lo que se pagó.

Cuántos drivers por iteración. Los que compartan objetivo. Una iteración puede cubrir varios drivers si una sola decisión los satisface o los incumple a la vez, y entonces separarlos obligaría a tomar la misma decisión dos veces; y un driver puede necesitar varias iteraciones si su enunciado exige decisiones sobre elementos distintos. Lo que no cambia es que la iteración tenga **un objetivo y un elemento a descomponer**, y que termine en un incremento que se pueda demostrar y medir.

1.6 Convenciones

- **Peso de una entrada.** Alto, medio o bajo. Alto significa que la entrada decide la estructura del elemento elegido: hay alternativas que la cumplen y otras que no. Medio, que la condiciona sin decidirla. Bajo, que se cumple dentro de cualquier alternativa razonable.
- **Entrada diferida.** No se descarta: se atiende en una iteración posterior, que se nombra en el momento de diferirla.
- **Concepto de diseño.** Estilo, patrón o táctica. Se nombra por lo que hace y no por una tecnología concreta; la elección de producto o biblioteca no pertenece a estos pasos.
- **Identificadores propios de este documento.** EL-..., el elemento que una iteración descompone; CD-..., los conceptos de diseño elegidos; C-..., los elementos que resultan de aplicarlos; R-..., los riesgos que quedan abiertos; ADR-..., las fichas de decisión. Los demás identificadores vienen de los documentos de entrada.

2. ASR

Un requisito arquitectónicamente significativo (ASR) es un requisito cuya satisfacción tiene un efecto profundo sobre la arquitectura del sistema, de manera que la arquitectura podría ser diferente si ese requisito no existiera. Esta sección identifica los ASR de Bastion, dice de dónde procede cada uno y los prioriza. Se construye solo a partir de los documentos de entrada.

2.1 Cómo se identifican

La prueba. A cada requisito candidato se le hace la misma pregunta: *¿la ausencia de este requisito podría llevarnos a diseñar una arquitectura significativamente diferente?* Si la respuesta es sí, se nombran las decisiones que obliga a tomar. Por ejemplo, recuperar una partida después de una falla del servidor en un máximo de 60 segundos es ASR porque obliga a decidir la persistencia del estado de la partida, cómo detectar el fallo, los mecanismos de recuperación, el despliegue y la sincronización. Si la respuesta es no, el requisito sigue siendo obligatorio, pero se cumple dentro de cualquier arquitectura razonable y no sirve para elegir entre ellas.

Las fuentes. La prueba se aplica a los candidatos de las tres fuentes de las que puede surgir un ASR, sin olvidar los objetivos de negocio, los stakeholders, los concerns y las dependencias externas que las respaldan.

Fuente	Qué se revisó
Funcionalidad arquitectónicamente significativa	Los 49 casos de uso del documento de casos de uso de Bastion. Solo cuentan los que, para realizarse, obligan a una decisión estructural: la mayoría se resuelve sobre la estructura que exigen otros, como se explica en la subsección 2.5.
Escenarios de calidad	Los 18 escenarios del documento de contexto, priorizados en su Utility Tree por importancia y sensibilidad arquitectónica.
Restricciones	Las restricciones técnicas y de proyecto (CON) y los requisitos impuestos (REQ) que eliminan alternativas. Incluye la organización del propio equipo: dos personas y 14 semanas.
Objetivos, stakeholders, concerns y dependencias	Los objetivos de negocio (juego en línea, público de 8 años en adelante, distribución internacional y originalidad frente a Quoridor), los 19 stakeholders con sus 26 concerns y las dependencias externas DEP-01 a DEP-04 y DEP-08. No son una fuente aparte: se usan para confirmar que el requisito es real y para medir su importancia.

2.2 Cómo se enuncia un ASR

Identificar el requisito no basta: hay que enunciarlo de una forma que sirva para diseñar. Un ASR dice **qué tiene que lograr el sistema**, no **cómo**. En cuanto el enunciado nombra un mecanismo, una pieza o una tecnología, deja de ser un requisito y se convierte en una decisión de diseño disfrazada de requisito, con dos consecuencias: la decisión entra al documento sin que nadie haya comparado alternativas, y el requisito ya no se puede usar para evaluar esas alternativas, porque solo una de ellas puede cumplirlo por definición.

Cada enunciado de esta sección se revisó contra tres reglas.

- **Describe un efecto observable**, no un componente. Si el enunciado nombra dónde vive algo o de qué está hecho, esa parte sobra.
- **Se puede comprobar** sobre el sistema terminado, sin conocer su interior. La medida acompaña al enunciado, no lo sustituye.
- **Deja abiertas las alternativas**. Si solo hay una manera de cumplirlo, el enunciado ya eligió por el diseñador.

La única excepción son las restricciones. Una restricción es, por definición, una decisión ya tomada por alguien con autoridad para tomarla, y se enuncia tal como llega; lo que se le exige entonces es la justificación de por qué es arquitectónicamente significativa, que aparece en la subsección 2.4.

Enunciados corregidos. Tres candidatos de la primera revisión traían la solución dentro. Se muestran aquí porque la corrección explica la forma que tienen hoy los quince enunciados.

Enunciado descartado	Qué era solución	Enunciado que quedó
Una jugada se responde en menos de 1 segundo, con mediana bajo 300 ms, <i>con el servidor decidiendo y el tráfico cifrado</i> .	Que decida el servidor y que el tráfico se cifre son dos decisiones de diseño. Además, mezclan tres requisitos distintos en un enunciado.	ASR-01, que conserva solo el tiempo de respuesta y su medida. Quién decide es ASR-02 y la protección del tránsito es ASR-03.

Enunciado descartado	Qué era solución	Enunciado que quedó
El estado de la partida debe proyectarse <i>de forma asimétrica</i> : el sistema <i>no envía</i> a un jugador la información oculta de otro.	La proyección asimétrica es el mecanismo. Lo exigible es el resultado: que nadie reciba lo que no le corresponde, se consiga como se consiga.	Parte de ASR-02: ningún participante obtiene información que no le corresponde, aunque use un cliente alterado.
El motor de reglas debe estar <i>aislado en una biblioteca sin dependencias de la interfaz ni de la red</i> , para poder ejecutarse en memoria.	El aislamiento en una biblioteca es una decisión de diseño; nombra incluso las piezas de las que no debe depender.	ASR-06: una partida completa debe poder ejecutarse y verificarse de forma automática, sin intervención manual, y los valores de las reglas deben cambiarse sin tocar el código.

Las soluciones que se retiraron de los enunciados no se perdieron: reaparecen más adelante como alternativas que se comparan entre sí, que es donde les corresponde decidirse.

2.3 ASR encontrados

Al aplicar la prueba a las tres fuentes se encontraron quince ASR: catorce activos y uno reservado. La tabla los reúne todos, con lo que la arquitectura debe cumplir, su procedencia y los elementos del análisis de los que vienen. El orden de la tabla es el de su numeración; el orden de importancia se establece en la subsección 2.6.

ASR	Qué debe cumplir la arquitectura	Procedencia	Viene de
ASR-01	El sistema responde a la acción de un jugador dentro de su turno en menos de 1 segundo, con una mediana por debajo de 300 ms, en el peor caso de operación previsto.	Escenario, Restricción	QAS-01; REQ-02; CRN-02; STK-01, STK-09; DEP-04.
ASR-02	El estado de una partida solo cambia por acciones legítimas de quien tiene el turno, y ningún participante obtiene información que no le corresponde, aunque use un cliente alterado.	Escenario	QAS-05; CRN-10, CRN-23; STK-15, STK-10, STK-05.
ASR-03	Lo que jugador y sistema intercambian no resulta legible ni reutilizable para un tercero que lo intercepte.	Escenario, Restricción	QAS-06; REQ-03, CON-02, CON-03; CRN-22, CRN-05; STK-15, STK-02; DEP-02.
ASR-04	Una interrupción no anunciada de la comunicación no deja la partida en un estado indefinido: se reconoce en menos de 15 segundos y, si el jugador vuelve antes de 2 minutos, continúa la misma partida en menos de 5 segundos, desde su equipo o desde otro.	Escenario, Funcionalidad	QAS-08; CU-26, CU-24; CON-17; CRN-06; STK-01, STK-14; DEP-04.
ASR-05	Una interrupción del almacenamiento no hace perder ningún dato ya confirmado al jugador ni más de 5 segundos de jugadas, y lo perdido queda identificado.	Escenario, Restricción	QAS-09; CON-04; CU-25; CRN-19; STK-13, STK-14; DEP-03, DEP-04.
ASR-06	Los valores que gobiernan las reglas del juego se modifican sin modificar ni recompilar el código, y una partida completa se ejecuta y se verifica de forma automática, sin intervención manual.	Escenario	QAS-12, QAS-17; CON-15; CRN-13, CRN-17; STK-06, STK-10.

ASR	Qué debe cumplir la arquitectura	Procedencia	Viene de
ASR-07	Reservado: dos jugadores de una misma red local juegan una partida completa sin depender de infraestructura externa.	Escenario, condicionado a Q-02	QAS-14; Q-02; CRN-20; STK-14.
ASR-08	Dos o más jugadores situados en equipos distintos juegan una partida completa, desde su creación por emparejamiento o por sala privada hasta su resultado definitivo, viendo todos el mismo estado y con el mismo turno y los mismos tiempos.	Funcionalidad, Restricción	REQ-01; CU-17, CU-18, CU-19, CU-20, CU-21, CU-25; objetivo de juego en línea; STK-03, STK-01.
ASR-09	Quien no participa en una partida puede seguirla sin conocer nada antes que los jugadores y sin afectar el tiempo de respuesta de estos, tenga cuenta o no.	Funcionalidad	CU-34, CU-12 (permite_espectadores), CU-21 (paso 20); CRN-02; STK-01.
ASR-10	Un jugador juega una partida completa contra un oponente automático, con distintos niveles de dificultad y con ayuda opcional, sin que el tiempo de respuesta comprometido deje de cumplirse.	Funcionalidad	CU-27; Q-01; CRN-03, CRN-14; STK-06, STK-04.
ASR-11	Ningún mensaje con contenido inapropiado llega a un jugador menor de edad, tampoco cuando el sistema no logra determinar si lo es; toda sanción se sustenta en evidencia y no se conservan conversaciones que nadie reportó.	Funcionalidad, Restricción, Escenario	CU-28, CU-33, CU-42 a CU-45; REQ-04, REQ-05; QAS-04, QAS-07; CRN-04, CRN-05, CRN-18; STK-02, STK-17, STK-19.
ASR-12	El acceso a una cuenta exige un segundo factor de verificación, y un niño de 8 años lo completa, solo o con apoyo de su tutor.	Restricción, Escenario	REQ-03, REQ-07; QAS-03; CU-01, CU-04; CRN-01, CRN-26; TEN-03; STK-15, STK-02; DEP-01, DEP-02.
ASR-13	El sistema se ofrece en un idioma adicional sin modificar el código, conserva íntegros los caracteres y los formatos de ese idioma en todo el recorrido del dato y no deforma sus pantallas cuando el texto es más largo.	Restricción, Escenario	REQ-06, CON-05, CON-04; QAS-11; CU-12; objetivo de distribución internacional; CRN-15, CRN-16; STK-11.
ASR-14	La solución se construye con C# sobre .NET, almacena sus datos en SQL Server y se comunica mediante sockets TCP, sin WebSockets.	Restricción	CON-01, CON-02, CON-03, CON-04, CON-16, CON-17; tecnología asignada por el cliente (STK-03).
ASR-15	La solución queda terminada en 14 semanas, construida por un equipo de dos personas, con un incremento verificable cada semana, y su alcance se reduce sin rehacer lo ya construido.	Restricción, Escenario	CON-13, CON-14; QAS-13, QAS-17; equipo de dos personas; CRN-07, CRN-08, CRN-14; STK-03, STK-04, STK-16.

Qué muestra la procedencia. Once ASR se apoyan en al menos un escenario, nueve en una restricción y cinco en una funcionalidad, y nueve combinan más de una fuente.

2.4 Por qué cada uno es ASR

Para cada ASR de la tabla anterior se responde la pregunta de la prueba: qué decisiones obliga a tomar y cómo podría ser la arquitectura si no existiera. Las soluciones que no pueden estar en el enunciado sí aparecen aquí, y como lo que son: las decisiones que el requisito pone sobre la mesa, todavía sin elegir.

ASR	¿Es ASR? Sí, porque obliga a decidir...	Sin este requisito, la arquitectura podría...
ASR-01	Cuántos viajes de red hay por jugada, cuánto trabajo cabe dentro del turno, qué sale del camino de la respuesta, cómo se reparte el segundo entre sus tramos y cómo se mide cada uno.	Validar cada jugada contra la base de datos, escribir de forma síncrona y notificar en secuencia: un diseño más simple y más lento.
ASR-02	Quién tiene la autoridad sobre el estado, dónde se ejecuta el motor de reglas, cómo se comprueba cada mensaje antes de tocar el estado y qué información lleva cada respuesta según quién la recibe.	Dejar que el cliente calcule y difunda el estado con el servidor como simple retransmisor, o jugar entre pares sin servidor de partidas.
ASR-03	Dónde se protege lo que viaja, sobre la conexión o mensaje por mensaje; cómo se gestionan certificados y claves; cómo es el apretón de manos; y cómo se impide que algo capturado se vuelva a usar.	Hacer viajar el protocolo propio en claro, sin apretón de manos ni vencimiento de sesiones.
ASR-04	Que la permanencia del jugador en la partida sea independiente de la conexión que lo trajo, cómo se reconoce una conexión medio abierta, cómo se reasocia al jugador desde otro equipo, dónde vive el estado mientras no está y qué reloj mide los plazos.	Ligar la partida al socket y darla por terminada cuando la conexión se cierra.
ASR-05	Qué se escribe de forma confirmada y qué de forma diferida, cuánto se puede perder, cómo se reconoce la pérdida del almacenamiento, qué ocurre con las partidas mientras no está y cómo se reconstruyen.	Escribir todo de forma síncrona o todo al final de la partida, sin cola, sin registro de lo perdido y sin suspensión.
ASR-06	Dónde viven los parámetros de las reglas, de qué depende el componente que las ejecuta y si su resultado es determinista.	Tener las reglas como constantes dentro del código del servidor, con el motor mezclado con el arbitraje y con la interfaz.
ASR-08	Que existan dos extremos y una comunicación entre ellos, quién crea y sostiene la partida, cómo se gobiernan turnos y relojes y dónde se conservan cuentas y resultados.	Ser un juego local en un solo equipo, sin red, sin servidor, sin protocolo y sin cuentas centralizadas. Es la diferencia más grande de toda la lista.
ASR-09	Cómo llega el estado a destinatarios que no juegan, cómo se evita que conozcan algo antes que ellos, cómo se admite a quien no tiene sesión sin abrir una vía de ataque y cómo se impide que su número frene a los jugadores.	Notificar a cada partida solo a sus participantes, todos autenticados, sin ningún mecanismo de difusión ni de retraso.
ASR-10	Dónde se ejecuta el oponente automático, cómo ocupa el lugar de un participante, cuánto de su cálculo cabe en el presupuesto por jugada y cómo se retira si el alcance se recorta.	Suponer siempre dos personas conectadas, sin nada que calcule jugadas por su cuenta.
ASR-11	Dónde se revisa el contenido; qué ocurre cuando esa revisión no responde; que el chat exceda a la partida, con canales global y de amigos; qué se conserva de los mensajes; y cómo se liga cada sanción a su evidencia.	Prescindir del servicio de chat y del de moderación, del catálogo de palabras prohibidas y de cualquier dato de menores que proteger.
ASR-12	Integrar un proveedor externo del segundo factor (DEP-02) y posiblemente el correo (DEP-01), gestionar secretos, dar a las cuentas estados propios, como pendiente de verificación, bloqueo temporal o aprobación del tutor, y aislar el mecanismo detrás de una interfaz mientras Q-03 siga abierta.	Resolver el acceso con usuario y contraseña, con un módulo de cuentas que no depende de ningún tercero.

ASR	¿Es ASR? Sí, porque obliga a decidir...	Sin este requisito, la arquitectura podría...
ASR-13	Convenciones que atraviesan todos los componentes: textos fuera del código, una codificación común de extremo a extremo, formatos por región y una interfaz sin anchos fijos. No cambia qué componentes existen, pero si se descubre tarde obliga a tocar cada pantalla, cada mensaje y cada columna.	Escribir los textos en el código, guardar los datos sin una codificación universal y maquetar las pantallas para un solo idioma.
ASR-14	Construir un protocolo propio sobre TCP, porque con WebSockets se heredarían la delimitación, el apretón de manos y la protección del tránsito; diseñar la persistencia sobre un modelo relacional; y compartir código entre los dos extremos en un mismo ecosistema.	Elegir libremente un protocolo estándar con esas capacidades ya resueltas, otro modelo de datos o un motor de juego en otro lenguaje. El protocolo dejaría de ser un componente a diseñar.
ASR-15	Construir por rebanadas verticales, verificar sin intervención manual, aislar solo lo que puede recortarse y mantener pocas piezas desplegables. Es la estructura de asignación de trabajo de Bass, Clements y Kazman: el tamaño del equipo condiciona en cuántas piezas conviene partir el sistema.	Plantearse servidores replicados, alta disponibilidad de la base de datos o muchos servicios desplegados por separado, que con más tiempo o más gente serían alternativas razonables.

ASR-07 no aparece en la tabla porque su prueba depende de Q-02: si se aprueba la LAN, obligará a separar el servidor de partidas y su configuración del cliente; si no, no existe.

2.5 Requisitos y decisiones que no son ASR

La prueba también sirve para descartar. Los siguientes elementos son obligatorios o importantes, pero su ausencia no llevaría a una arquitectura significativamente diferente: se cumplen sobre la estructura que los ASR ya imponen.

Elemento	Fuente	Por qué no pasa la prueba
Perfil, nickname y cosméticos (CU-12 a CU-16)	Funcionalidad	Son lecturas y escrituras sobre la cuenta. Cualquier arquitectura que cumpla ASR-08 los resuelve sin cambiar ningún componente.
Tienda, cajas y monedas (CU-38 a CU-40)	Funcionalidad	Necesitan transacciones, que SQL Server ya ofrece y que ASR-05 ya obliga a tratar como escritura confirmada. No piden ninguna decisión estructural nueva.
Amigos e invitaciones (CU-30 a CU-32, CU-49)	Funcionalidad	Usan la misma comunicación y la misma persistencia que ASR-08; el canal de amigos del chat ya entra en ASR-11.
Ranking e historial (CU-35, CU-36)	Funcionalidad	Son consultas sobre datos que otros ASR ya obligan a guardar.
Repetición de una partida (CU-37)	Funcionalidad	Se apoya en el determinismo de ASR-06 y en las jugadas registradas de ASR-05; no añade una decisión propia.
Tutorial (CU-41)	Funcionalidad	Es importante para REQ-07, pero se resuelve dentro de los componentes que otros ASR ya exigen.
Moderadores y bitácoras (CU-46 a CU-48)	Funcionalidad	Son operaciones administrativas sobre datos que ASR-02 y ASR-11 ya obligan a registrar.
Originalidad frente a Quoridor y licencias de recursos (CON-15, CON-18; QAS-15)	Restricción	Importancia alta y sensibilidad baja: se cumplen con marca, textos y recursos fuera del código, una convención que ASR-13 ya impone. Ninguna estructura razonable lo impide.

Elemento	Fuente	Por qué no pasa la prueba
Instalación en plataformas de distribución (Q-10, QAS-18, DEP-08)	Restricción condicionada	Solo aplica si se publica, y se resuelve con el empaquetado, sin tocar componentes.
DEC-01: modelo cliente-servidor basado en servicios	Decisión del equipo	No es un requisito, sino la respuesta del equipo a ASR-08 y ASR-02. Se registra aquí como decisión ya tomada, para que el diseño parta de ella.
DEC-02: no se confía en el cliente; toda la lógica del juego se ejecuta en el servidor	Decisión del equipo	Responde a ASR-02 de la forma más estricta posible. Condiciona cómo se cumplen ASR-01 y ASR-06, pero no es un requisito.
DEC-03: una partida en línea exige conexión con la base de datos	Decisión del equipo	Fija cómo se cumple ASR-05: ante una caída, la partida no continúa sin base de datos. Es una decisión sobre el requisito, no un requisito nuevo.

2.6 Priorización

Ya confirmado el conjunto, falta decidir qué ASR pesa más. La priorización sigue el mismo esquema que el Utility Tree, con dos criterios independientes que se califican por separado y se combinan en un nivel de prioridad.

Cómo se prioriza. La *importancia* mide cuánto le importa el requisito a quien puede decidir sobre el proyecto; la *sensibilidad arquitectónica* mide cuánto cambia la estructura según se cumpla o no. La escala de cada criterio es la siguiente.

Nivel	Importancia	Sensibilidad arquitectónica
Alta (H)	Lo impone el cliente, la regulación o un requisito impuesto, o su incumplimiento provoca el rechazo de un stakeholder con poder de decisión.	Hay alternativas estructurales razonables y solo algunas lo cumplen, y cambiar de alternativa después afecta a varios componentes a la vez.
Media (M)	Depende de una pregunta abierta o de una funcionalidad valiosa pero no esencial.	Su efecto queda dentro de un componente, o en convenciones que atraviesan el sistema sin cambiar su partición.
Baja (L)	Es deseable, pero ningún stakeholder lo exige.	Se cumple igual en cualquier alternativa razonable.

Las dos calificaciones se combinan en cuatro niveles. Un ASR (H, H) es **prioritario**: es un criterio de descarte, y una alternativa que no pueda cumplirlo se rechaza aunque sea mejor en todo lo demás. Un ASR (H, M) tiene prioridad **alta**: es obligatorio, pero se cumple dentro de la estructura que los prioritarios imponen. Un ASR (M, M) tiene prioridad **media**. Un ASR con importancia media y sensibilidad alta queda **reservado** mientras dependa de una pregunta abierta, porque tomarlo en cuenta hoy obligaría a diseñar para una situación que puede no llegar. Ningún ASR resultó con calificación baja: un requisito así no habría pasado la prueba. Dentro de cada nivel, el orden sigue a las dependencias: primero va lo que los demás dan por hecho.

ASR priorizados. La tabla ordena los quince ASR con la justificación de cada una de sus dos calificaciones.

Orden	ASR	Importancia y por qué	Sensibilidad y por qué	Prioridad
1	ASR-08	H. Lo impone el cliente (REQ-01) y es el objetivo central del juego.	H. Sin él no hay dos extremos, ni comunicación, ni datos compartidos: es la diferencia estructural más grande de la lista.	Prioritario
2	ASR-02	H. Lo exige STK-15, que da por hecho un cliente alterado (CRN-23).	H. Separa un extremo que decide de uno que solo retransmite, y ese cambio rehace cliente, servidor y protocolo.	Prioritario
3	ASR-14	H. La tecnología la asigna el cliente (STK-03) y no se negocia.	H. Obliga a un protocolo propio y a un modelo relacional, que otras tecnologías darían resueltos.	Prioritario
4	ASR-03	H. Sin él, el segundo factor de REQ-03 no protege nada y el chat de menores viaja legible (STK-15, STK-02).	H. Dónde se protege lo que viaja cambia el formato de los mensajes en los dos extremos.	Prioritario
5	ASR-04	H. Los jugadores son niños en redes domésticas inestables (CRN-06, STK-01).	H. Obliga a que la permanencia en la partida no dependa de la conexión, lo que toca servidor, protocolo y cliente.	Prioritario
6	ASR-01	H. Es el único requisito con un número impuesto (REQ-02).	H. Su reparto cambia el flujo de la jugada en todos los componentes que atraviesa.	Prioritario
7	ASR-05	H. Lo exige STK-13 (CRN-19), sobre un almacenamiento alojado por un tercero (DEP-03, DEP-04).	H. Decide qué componente tiene el estado vigente y cuándo se escribe.	Prioritario
8	ASR-06	H. El diseñador cambiará las reglas decenas de veces (CRN-13) y CON-15 puede obligar a variantes.	H. Decide de qué depende el componente que ejecuta las reglas y si su resultado se puede reproducir.	Prioritario
9	ASR-11	H. Lo imponen REQ-04 y REQ-05, y STK-02 puede vetar el juego.	M. Su efecto queda en el chat y la moderación, una vez decidido quién tiene la autoridad.	Alta
10	ASR-12	H. Lo imponen REQ-03 y REQ-07.	M. Queda dentro del módulo de cuentas y de sus dependencias externas.	Alta
11	ASR-13	H. Lo imponen REQ-06 y el objetivo de distribución internacional.	M. Impone convenciones transversales, no una partición distinta.	Alta
12	ASR-15	H. El plazo y los incrementos los fijan el cliente y el auditor (STK-03, STK-16).	M. Acota las alternativas sin elegir entre las razonables.	Alta
13	ASR-09	M. Es funcionalidad valiosa pero no esencial.	M. Pide un mecanismo de difusión dentro del servidor.	Media
14	ASR-10	M. CU-27 lo especifica, pero Q-01 sigue abierta en el documento de contexto.	M. El oponente automático entra como un participante más dentro de la estructura de partidas.	Media
15	ASR-07	M. Depende de Q-02.	H. Si se aprueba, obliga a separar el servidor y su configuración.	Reservado

Por qué este orden entre los prioritarios. ASR-08 va primero porque sin él no hay partida en línea que sostener; ASR-02 decide de quién depende que esa partida sea legítima; ASR-14 fija con qué se construye todo. Los cinco restantes piden comportamientos que solo tienen sentido una vez que los tres primeros existen: la protección de lo que viaja y la supervivencia a una desconexión se apoyan en el canal que ASR-14 obliga a construir, y el tiempo de respuesta, la persistencia y las reglas, sobre la partida que ASR-08 y ASR-02 ya establecieron.

2.7 Resultado

Quedan catorce ASR activos y uno reservado. Los ocho prioritarios, ASR-01 a ASR-06, ASR-08 y ASR-14, son el criterio de descarte de cualquier alternativa de diseño. Los cuatro de prioridad alta, ASR-11 a ASR-13 y ASR-15, son obligatorios pero se cumplen dentro de la estructura que los prioritarios imponen. Los dos de prioridad media, ASR-09 y ASR-10, dependen de funcionalidades no esenciales o de una pregunta abierta. Junto con las tres decisiones del equipo, este conjunto es la entrada de todo lo que sigue en el documento.

3. Drivers

Un driver arquitectónico es el problema de diseño que la arquitectura tiene que resolver, enunciado de forma que sirva para conducir una iteración. Esta sección toma los cinco ASR de mayor prioridad de la subsección 2.6, los órdenes 1 a 5, y los convierte en cinco drivers.

3.1 Cómo se convierte un ASR en driver

Un driver no es un ASR con otro nombre. Es el mismo problema visto desde arriba: el ASR dice qué hay que cumplir y cuánto; el driver dice qué propiedad del sistema está en juego y, por lo tanto, qué hay que decidir. Cada conversión siguió cuatro reglas.

Regla	Qué significa al escribir el driver
Más general que el ASR	El driver enuncia la propiedad; la cifra que la comprueba se queda en el ASR y en su escenario. Generalizar no es volverse vago: el driver sigue diciendo qué tiene que ocurrir y de qué sistema se habla, y se puede rastrear hasta una medida concreta.
Sin soluciones	Se aplica el mismo criterio de la subsección 2.2. Si el enunciado nombra un componente, un mecanismo o una tecnología, esa parte se retira: es lo que la iteración tiene que decidir comparando alternativas. Un driver que ya trae la respuesta no conduce ninguna iteración.
Las restricciones se enuncian tal cual	Una restricción ya es una decisión tomada por quien tiene autoridad para tomarla, y no hay nada que generalizar. Se copia como llega y se le agrega, en la última columna, por qué es arquitectónicamente significativa: qué alternativas elimina y qué obliga a construir.
Un driver, un ASR de origen	Cada driver se rastrea hasta un solo ASR, y conserva su lugar en el orden. Así, cuando un ASR cambia, se sabe exactamente qué driver revisar.

3.2 Los cinco drivers

Cada fila recoge el enunciado del driver, de qué tipo de entrada procede, el ASR del que se deriva con su lugar en el orden de la subsección 2.6 y por qué ese driver obliga a decidir algo que cambia la arquitectura.

Driver	Enunciado	Tipo	ASR de origen	Por qué es arquitectónicamente significativo
DRV-01	El sistema sostiene el desarrollo completo de una partida entre jugadores que están en equipos distintos, de modo que todos los participantes vean el mismo juego.	Funcionalidad primaria	ASR-08 (orden 1)	Es el driver que obliga a que el sistema tenga más de un extremo. De él dependen la existencia de una comunicación, de un lugar donde vive lo común a todos los participantes y de un almacenamiento compartido. Sin él, Bastion sería un programa en un solo equipo y ninguno de los otros cuatro drivers tendría sentido.
DRV-02	El desarrollo y el resultado de una partida dependen únicamente de acciones legítimas, y cada participante conoce solo lo que le corresponde, sin suponer que su equipo sea confiable.	Atributo de calidad: seguridad (integridad y confidencialidad)	ASR-02 (orden 2)	Obliga a decidir de qué extremo depende que una partida sea correcta y dónde se ejecutan las reglas. Es la decisión que CRN-10 describe como la más cara de cambiar: si se cambia después, cada mensaje cambia de significado, porque deja de ser una propuesta y pasa a ser un hecho, y hay que rehacer los dos extremos a la vez.
DRV-03	La solución se construye con C# sobre .NET, almacena sus datos en SQL Server y se comunica mediante sockets TCP, sin WebSockets.	Restricción	ASR-14 (orden 3)	Elimina las alternativas que traen resuelto el transporte. Sin WebSockets, la delimitación de los mensajes, el versionado, el apretón de manos y la protección de lo que viaja pasan a ser componentes que el equipo diseña y mantiene. SQL Server fija un modelo relacional para todo lo que se conserve, y un único ecosistema permite que los dos extremos compartan código. Es una decisión ya tomada, pero condiciona todas las demás.
DRV-04	Lo que el jugador y el sistema intercambian no resulta aprovechable para un tercero que lo observe o lo repita.	Atributo de calidad: seguridad (confidencialidad)	ASR-03 (orden 4)	Afecta a los dos extremos al mismo tiempo y al formato de todos los mensajes, y su costo se descuenta del tiempo de respuesta comprometido. Decidirlo tarde obliga a rehacer el protocolo ya construido y a cambiar los clientes ya distribuidos.
DRV-05	La partida y la posición del jugador dentro de ella sobreviven a la interrupción de su comunicación y a un cambio de equipo.	Atributo de calidad: fiabilidad (recuperabilidad)	ASR-04 (orden 5)	Obliga a que la permanencia del jugador en la partida no dependa del canal que lo trajo, y a que lo común a la partida siga existiendo mientras él no está. Es lo que separa una arquitectura donde la partida vive atada a una conexión de otra donde la conexión es solo el medio por el que se llega a ella.

Qué se generalizó y qué se retiró. La tabla siguiente deja a la vista la conversión, para que se pueda comprobar regla por regla.

Driver	Qué se generalizó respecto al ASR	Qué se retiró por ser solución
DRV-01	Las cifras de ASR-08, la forma de crear la partida por emparejamiento o por sala y el detalle de los turnos se quedan en el ASR. El driver conserva lo que de verdad conduce el diseño: jugadores en equipos distintos y un solo juego para todos.	Nada quedaba por retirar: el enunciado corregido de ASR-08 ya no nombraba ningún componente.
DRV-02	ASR-02 habla del estado de la partida y del turno; el driver habla del desarrollo y del resultado, que es la propiedad que el jugador percibe y que abarca también el final de la partida.	Del primer borrador se retiraron “el servidor decide” y “proyección asimétrica del estado”. Cuál de los dos extremos tiene la autoridad es justamente lo que decide la iteración 1.
DRV-03	No se generaliza: es una restricción y se enuncia como llegó.	Nada. Lo que se agregó es la justificación de su significancia, que la regla exige para las restricciones.
DRV-04	ASR-03 enumera credenciales, códigos, sesiones y chat; el driver habla de todo lo que se intercambia, y añade “ni repetirlo”, que en ASR-03 aparecía como no reutilizable.	Se retiró el cifrado. Cifrar es la solución previsible, pero no la única, y no puede estar en el enunciado del problema.
DRV-05	ASR-04 fija 15 segundos, 2 minutos y 5 segundos; el driver dice qué tiene que sobrevivir, que es lo que obliga a decidir. Las cifras vuelven cuando esa iteración tome QAS-08 como medida.	Se retiraron la sesión separada de la conexión y la detección por latidos, que eran dos decisiones metidas dentro del requisito.

3.3 Prioridad de los drivers

La prioridad de cada driver es la de su ASR de origen, tal como la fijó la subsección 2.6. A ese orden se le añade una comprobación: que sea viable para construir, porque un driver no se puede abordar antes que aquel del que depende.

Orden	Driver	ASR (orden)	Por qué ocupa ese lugar y de qué depende
1	DRV-01	ASR-08 (1)	Prioritario, y además no depende de ningún otro: es el que crea los dos extremos sobre los que los demás deciden algo. Los otros cuatro drivers hablan de una partida en línea que solo existe si este se resuelve.
2	DRV-02	ASR-02 (2)	Prioritario. Depende de que existan dos extremos (DRV-01) y de nada más. Va antes que los tres siguientes porque decide dónde se ejecutan las reglas, y eso cambia qué mensajes existen, cuánto trabajo hay dentro del turno y qué hay que proteger.
3	DRV-03	ASR-14 (3)	Prioritario y, por ser restricción, presente desde la primera iteración: no se “resuelve” en un turno, sino que acota lo que las demás pueden elegir. Ocupa este lugar porque lo que obliga a construir, el protocolo propio, es la entrada de DRV-04 y de DRV-05.
4	DRV-04	ASR-03 (4)	Prioritario. Depende de DRV-03, que es el que deja el transporte sin protección heredada, y de DRV-02, que define qué mensajes hay que proteger.
5	DRV-05	ASR-04 (5)	Prioritario. Es el último de los cinco porque necesita las tres decisiones anteriores: no se puede decidir cómo vuelve un jugador a su partida sin saber dónde vive lo que tiene que encontrar (DRV-02), por qué canal llega (DRV-03) y cómo se comprueba que sigue siendo él (DRV-04).

El orden heredado coincide con el orden construible. Ningún driver aparece antes que otro del que depende, así que no hace falta reordenar.

3.4 Lo que queda fuera de estos cinco drivers

Los nueve ASR activos restantes, del orden 6 al 14, y el reservado ASR-07 no producen driver en esta ronda. No desaparecen: siguen siendo entradas del diseño, y tres de ellos condicionan ya la primera iteración.

ASR	Situación	Qué papel cumple mientras tanto
ASR-01	Orden 6, prioritario. Driver en la siguiente ronda.	Entra en la iteración 1 como límite: ninguna decisión puede impedir que una jugada se responda en menos de un segundo. No se reparte el presupuesto todavía, pero sí se descartan las alternativas que lo agotan.
ASR-05	Orden 7, prioritario. Driver en la siguiente ronda.	Entra en la iteración 1 como condición, a través de DEC-03: la partida exige almacenamiento disponible. Qué se escribe y cuándo se decide después.
ASR-06	Orden 8, prioritario. Driver en la siguiente ronda.	Entra en la iteración 1 como condición: la decisión sobre dónde se ejecutan las reglas no puede impedir que después se parametricen y se verifiquen solas.
ASR-11 a ASR-13, ASR-15	Órdenes 9 a 12, prioridad alta.	Son obligatorios, pero se cumplen dentro de la estructura que los cinco drivers imponen. ASR-15 actúa además como filtro permanente: una alternativa que dos personas no puedan construir en 14 semanas se descarta en cualquier iteración.
ASR-09, ASR-10	Órdenes 13 y 14, prioridad media.	Solo exigen que las decisiones de ahora no les cierren la puerta: que se pueda notificar a alguien que no juega y que un oponente automático pueda ocupar el lugar de un participante.
ASR-07	Reservado.	No entra hasta que Q-02 se resuelva.

4. Iteración 1 — Partida en línea íntegra

4.1 Alcance de la iteración

Elemento	Descripción
Drivers que cubre	DRV-01 y DRV-02, los dos primeros de la sección 3. DRV-03, por ser restricción, está presente sin ser objetivo: acota lo que se puede elegir.
Por qué los dos juntos	Comparten el mismo objetivo y no se pueden comprobar por separado. DRV-01 pide que una partida entre equipos distintos llegue hasta el final; DRV-02 pide que ese final dependa solo de acciones legítimas. Una partida que se completa pero que un cliente alterado puede torcer no cumple ninguno de los dos, y una partida íntegra que no se puede terminar tampoco. Además, las dos alternativas en juego son las mismas: quién de los dos extremos produce el juego. Separarlos obligaría a tomar la misma decisión dos veces.
Qué decide	La estructura general del sistema: en qué partes se divide, de qué extremo depende que una partida sea correcta y dónde vive lo que la partida tiene en común mientras dura.
Qué no decide	Cómo viaja un mensaje: la delimitación, el versionado y el apretón de manos son de DRV-03 y se deciden en la iteración siguiente. Tampoco decide cómo se protege lo que viaja (DRV-04), cómo vuelve un jugador después de un corte (DRV-05), el reparto del segundo por jugada (ASR-01), qué se escribe y cuándo (ASR-05) ni cómo se parametrizan las reglas (ASR-06).
Decisiones previas	El equipo ya tomó DEC-01, DEC-02 y DEC-03, registradas en la sección 2. Entran como candidatas con respaldo, no como hechos incuestionables: esta iteración es donde se comparan con sus alternativas y quedan registradas con su razonamiento en las fichas ADR-001 a ADR-003.
Con qué termina	Con las fichas de decisión de la subsección 4.6. La instanciación completa de los elementos, sus interfaces y el análisis de la iteración quedan para los pasos siguientes del método.

4.2 Paso 1. Revisión de entradas

Se revisan las cinco entradas del método y se listan las que tienen algo que ver con los dos drivers. Las demás siguen siendo válidas, pero no dicen nada sobre cómo se reparte el sistema ni sobre de qué depende que una partida sea correcta.

Drivers de la iteración

Driver	Enunciado	ASR (orden)	Papel en esta iteración
DRV-01	El sistema sostiene el desarrollo completo de una partida entre jugadores que están en equipos distintos, de modo que todos los participantes vean el mismo juego.	ASR-08 (1)	Objetivo. Aporta el incremento: una partida que llega hasta el final.
DRV-02	El desarrollo y el resultado de una partida dependen únicamente de acciones legítimas, y cada participante conoce solo lo que le corresponde, sin suponer que su equipo sea confiable.	ASR-02 (2)	Objetivo. Aporta la medida: QAS-05.
DRV-03	La solución se construye con C# sobre .NET, almacena sus datos en SQL Server y se comunica mediante sockets TCP, sin WebSockets.	ASR-14 (3)	Restricción presente. No se decide aquí, pero elimina alternativas: no hay transporte con sesión, apretón de manos ni protección heredados.

Objetivos de diseño

ID	Objetivo	Origen y qué exige aquí
OBJ-01	Dos jugadores pueden jugar una partida completa en línea, de principio a fin.	REQ-01 y el objetivo de negocio de juego en línea. Es lo que vuelve obligatorio que el sistema tenga más de un extremo.
OBJ-02	Nadie puede alterar una partida desde un equipo que controla.	CRN-23 y el interés de STK-15. Da por hecho que el cliente es hostil, y eso decide de quién depende que la partida sea correcta.
OBJ-03	El juego resulta parejo para los dos jugadores, sin ventajas que dependan del equipo de cada uno.	CRN-02 y CRN-12, con STK-01 y STK-09. Exige que ninguno de los dos vea antes ni distinto, aunque su equipo sea más rápido.
OBJ-04	Cada semana hay un incremento funcional y demostrable dentro de las 14 semanas.	CON-13 y CON-14. Limita lo que esta iteración puede permitirse: la estructura tiene que poder construirse por partes y probarse desde el primer incremento.

Requisitos funcionales primarios

Del documento de casos de uso se toman como primarios los que cambian el estado de una partida en curso o dependen de él. Son los que obligan a arbitrar y los que vuelven visible cualquier discrepancia entre lo que ve el jugador y lo que el sistema acepta.

CU	Caso de uso	Qué exige de esta iteración
CU-20	Mover el peón	Una de las dos acciones del turno. Antes de aceptarla hay que comprobar sesión, participación, turno, número de jugada y reloj, y el resultado tiene que llegar igual a todos los participantes.

CU	Caso de uso	Qué exige de esta iteración
CU-21	Colocar un muro	La otra acción del turno y la más cara: exige comprobar que el muro no se solape ni se cruce (RN-04) y que todos los jugadores activos conserven camino a su meta (RN-05), sobre el tablero vigente. Es donde más se nota quién hace ese cálculo.
CU-22, CU-23	Ofrecer y aceptar tablas; rendirse	Terminan la partida por acuerdo o por decisión de un jugador. Una oferta pendiente caduca cuando llega una jugada (RN-05 del CU-22), así que dependen del mismo árbitro que las jugadas.
CU-24	Abandonar la partida	Distingue entre irse y desconectarse: a los 60 segundos de desconexión el rival puede reclamar la victoria (RN-03 del CU-26), lo que exige que el tiempo lo mida el sistema y no el cliente.
CU-25	Finalizar la partida y aplicar la progresión	Es el único momento en que el resultado se vuelve definitivo: forma de término, resultado por participación, elo, progresión y monedas. Cierra el recorrido del incremento.
CU-17, CU-18, CU-19	Buscar partida clasificatoria; crear sala privada; unirse con código	Crean la partida y fijan sus parámetros. No se descomponen aquí, pero determinan el momento en que la partida empieza a existir.

Quedan fuera de los primarios y se revisan en iteraciones posteriores: CU-26 (reconectar), que es el caso de DRV-05; CU-27 (jugar contra la IA), que depende de Q-01; CU-28 y CU-29 (chat y emotes), que dependen de ASR-11; CU-34 (ver una partida como espectador), que consume el estado pero no lo cambia; y CU-36 y CU-37 (historial y repetición), que dependen de lo que se decida persistir.

Escenarios de atributos de calidad

Escenario	Atributo	Qué exige de esta iteración
QAS-05 (ASR-02)	Seguridad: integridad	Es la medida del objetivo. Con una batería de 200 mensajes inválidos de un cliente modificado, 50 de ellos aleatorios: el 100 % se rechaza, el estado de la partida es idéntico antes y después de cada mensaje, 0 respuestas contienen datos privados del rival, el sistema no se detiene, las otras 20 partidas siguen cumpliendo QAS-01 y el 100 % de los intentos queda registrado con cuenta y hora.
QAS-01 (ASR-01)	Desempeño: comportamiento temporal	No es el objetivo, pero sí el límite. Ninguna decisión puede impedir que, en 1 000 colocaciones de barrera, el 100 % responda en menos de 1 segundo y la mediana quede bajo 300 ms, con el peor tablero y 20 partidas simultáneas.
QAS-07	Responsabilidad: trazabilidad	Cada acción tiene que quedar ligada a la cuenta que la hizo. Se apoya en el mismo registro que pide QAS-05.
QAS-08 (ASR-04)	Fiabilidad: recuperabilidad	No se atiende aquí, pero condiciona: la estructura no puede hacer que la partida deje de existir cuando se cierra una conexión, porque entonces DRV-05 sería irrealizable sin rehacerla.
QAS-09 (ASR-05)	Fiabilidad y responsabilidad	No se atiende aquí. Condiciona en un punto: tiene que quedar claro de dónde se reconstruye una partida si el almacenamiento se cae.
QAS-12 (ASR-06)	Modificabilidad	No se atiende aquí. Condiciona en un punto: dónde se ejecuten las reglas no puede impedir que después se parametricen y se verifiquen sin intervención manual.

Restricciones

ID	Restricción	Cómo limita esta iteración
REQ-01	Modo multijugador en línea obligatorio	La partida no puede resolverse en un solo equipo: hay dos extremos y una red entre ellos.
CON-02	La comunicación es por sockets TCP	Hay entrega ordenada y confiable, pero también conexiones medio abiertas: una caída de Wi-Fi no cierra el socket.
CON-03	Está prohibido usar WebSockets	Lo que otro transporte traería resuelto lo construye el equipo. Ninguna alternativa puede apoyarse en una sesión ni en un apretón de manos que vengan dados.
CON-17	La pila TCP/IP varía entre entornos	Impide confiar en el mecanismo del sistema operativo para reconocer una conexión caída: tarda horas y no es uniforme.
CON-04, DEP-03	Almacenamiento en SQL Server, alojado en infraestructura externa	El almacenamiento puede caerse y su latencia no está bajo el control del equipo.
DEP-04	Infraestructura de servidores y red provista externamente	El proceso puede reiniciarse sin aviso; la recuperación sí es responsabilidad del equipo.
CON-13, CON-14	Plazo de 14 semanas e incrementos auditables	Descartan las alternativas que no se puedan construir y probar por partes dentro del plazo.
CON-01, CON-16	C# sobre .NET	Acotan herramientas, no estructura. Abaratan que los dos extremos compartan un mismo ensamblado.

Decisiones de proyecto ya tomadas

ID	Decisión	Qué aporta a esta iteración
DEC-01	El sistema sigue un modelo cliente-servidor basado en servicios.	Entra como la alternativa que el equipo prefiere para repartir el sistema. Se compara en ADR-001 y ADR-002.
DEC-02	No se confía en el cliente: toda la lógica del juego se ejecuta en el servidor.	Entra como la alternativa más estricta frente a OBJ-02. Se compara en ADR-001 y ADR-003. Si se acepta, el cliente no valida reglas ni calcula resultados, ni siquiera para mostrarlos.
DEC-03	Una partida en línea exige conexión con la base de datos.	Condiciona ADR-004: la partida no se juega sin almacenamiento disponible, tal como exige PRE-04 de CU-20, CU-21 y CU-25. Qué se escribe y cuándo se decide en una iteración posterior.

Concerns

ID	Concern y qué aporta a la iteración
CRN-10	<i>"Tengo que decidir quién manda sobre el estado de la partida, el cliente o el servidor. De esa decisión cuelga casi todo lo demás, y cambiarla más adelante me sale carísimo."</i> Es el concern que da origen a DRV-02.
CRN-23	El especialista en ciberseguridad da por hecho que alguien modificará el cliente y probará con uno alterado. Fija el supuesto de confianza con el que se diseña.
CRN-12	El equipo teme que cliente y servidor discrepen sobre si una jugada es válida y que el jugador vea una jugada aceptada que después se revierte.
CRN-02	El jugador espera respuesta inmediata al colocar una barrera. Es el contrapeso de todo lo anterior.
CRN-11	Sin WebSockets hay que definir dónde empieza y termina cada mensaje y qué hacer cuando los dos extremos no tienen la misma versión.
CRN-07	El equipo, de dos personas, no puede sostener muchas piezas desplegadas a la vez.

Preguntas abiertas que condicionan la iteración

ID	Cómo condiciona
Q-01	Si habrá un oponente controlado por IA. Afecta a quién ocupa el lugar del segundo jugador, no a de quién depende la partida. La estructura elegida no debe impedir que ese oponente entre como un participante más.
Q-02	Si habrá conexión LAN. No cambia de quién depende la partida, pero obliga a que la parte que arbitra sea desplegable por separado y a que su dirección se lea de configuración.
Q-09	Con qué mecanismo se protege lo que viaja. Se decide en DRV-04; aquí solo importa que cargará un costo al presupuesto de QAS-01.

4.3 Paso 2. Objetivo de la iteración y selección de entradas

Priorización de las entradas

Cada entrada del paso 1 se pesa por lo que aporta a este objetivo y no por su importancia general: una entrada puede ser crítica para el proyecto y pesar poco aquí si cualquier alternativa razonable la cumple.

Entrada	Peso	Por qué pesa lo que pesa	Decisión
QAS-05 (ASR-02)	Alto	Es la entrada que decide entre las alternativas: con el cliente produciendo el juego, la medida es imposible; con el sistema produciéndolo, es alcanzable.	Se atiende
CU-20, CU-21	Alto	Son las dos acciones que cambian la partida. Sus comprobaciones definen qué hay que saber en el momento de aceptar o rechazar.	Se atiende
CRN-10	Alto	Enuncia la decisión y advierte su costo de cambio. Es el motivo por el que la iteración existe.	Se atiende
OBJ-01, OBJ-02	Alto	Son los dos objetivos que los drivers traducen. Juntos descartan tanto el juego local como el juego con un extremo confiado.	Se atiende
CON-02, CON-03, CON-17	Alto	Determinan que nada del transporte viene dado y que una conexión caída no se nota por sí sola.	Se atiende como restricción
DEC-01, DEC-02	Alto	Son las alternativas preferidas por el equipo y hay que compararlas con las demás antes de registrarlas.	Se atiende
QAS-01 (ASR-01)	Alto, como límite	No es el objetivo, pero descarta toda alternativa que agregue viajes dentro del turno o que espere a un tercero antes de responder.	Se atiende como límite
DEC-03	Medio	Convierte la disponibilidad del almacenamiento en condición de la partida. Aquí solo fija que la partida no sigue a ciegas si se pierde.	Se atiende como condición
CU-22 a CU-25	Medio	Aportan las formas de término, que la partida tiene que poder alcanzar para que el incremento esté completo.	Se atiende
CRN-23, CRN-12	Medio	Son el origen de QAS-05: fijan el supuesto de cliente hostil y el temor a que los dos extremos discrepen.	Se atiende
QAS-07	Medio	Pide un registro que QAS-05 ya exige con más detalle.	Se atiende por vía indirecta
QAS-08 (ASR-04)	Alto en su iteración, condición aquí	Obliga a que la partida no muera con la conexión. Decidir cómo vuelve el jugador necesita el protocolo, que no existe todavía.	Se difiere a DRV-05

Entrada	Peso	Por qué pesa lo que pesa	Decisión
QAS-09 (ASR-05)	Alto en su iteración, condición aquí	Obliga a definir qué se pierde ante una caída del almacenamiento. Necesita decidir sobre el acceso a datos.	Se difiere
QAS-12 (ASR-06)	Bajo aquí	Que las reglas se parametricen y se verifiquen solas es otro objetivo. Aquí solo hay que dejarle sitio.	Se difiere
CU-17, CU-18, CU-19	Bajo	Crean la partida, pero cualquier forma de crearla es compatible con las alternativas en juego.	Se difiere
CRN-11	Bajo aquí	Delimitación y versionado son del protocolo: condicionan el costo por mensaje, no de quién depende la partida.	Se difiere a DRV-03
CON-04, DEP-03, DEP-04	Medio	Fijan qué puede fallar, no cómo se estructura la respuesta a la falla.	Se atiende como supuesto
CON-13, CON-14, CRN-07	Medio	No eligen entre alternativas; descartan las que no se puedan construir por partes ni sostener entre dos personas.	Se atiende como filtro
Q-01, Q-02	Bajo	Ninguna cambia de qué depende la partida. Solo imponen que un oponente automático pueda entrar como participante y que la parte que arbitra sea desplegable por separado.	Se atiende como condición

Objetivo de la iteración

Elemento	Descripción
Objetivo	Que dos jugadores en equipos distintos puedan llevar una partida de principio a fin y que su desarrollo dependa únicamente de acciones legítimas: ningún mensaje, venga de donde venga, cambia la partida si no es una jugada legal del participante a quien le toca el turno.
Drivers que cubre	DRV-01, en la parte del enunciado que pide una partida completa entre equipos distintos con un mismo juego para todos. DRV-02, completo.
ASR que satisface	ASR-08 y ASR-02, órdenes 1 y 2 de la subsección 2.6.
Medida	La de QAS-05: con 200 mensajes inválidos de un cliente modificado, 50 de ellos aleatorios, el 100 % se rechaza, el estado de la partida es idéntico antes y después de cada mensaje, 0 respuestas llevan datos privados del rival y el 100 % de los intentos queda registrado con cuenta y hora.
Límite que respeta	QAS-01. Ninguna decisión de esta iteración puede impedir que una jugada se responda en menos de un segundo, con mediana bajo 300 ms; el reparto del presupuesto entre los tramos se decide después.
Incremento que produce	Dos clientes juegan una partida completa en línea: CU-20 y CU-21 funcionan de principio a fin, la partida llega a alguna de las formas de término de CU-22 a CU-25, y un cliente de prueba que envía mensajes inválidos no consigue alterarla. Es demostrable al cerrar la iteración y sirve como incremento semanal (CON-14).
Criterio de éxito	La iteración termina cuando ese incremento pasa la medida de QAS-05 sin violar QAS-01. Una alternativa de diseño que no lo permita se descarta, aunque sea mejor en todo lo demás.

4.4 Paso 3. Elemento a descomponer

Este es el elemento sobre el que trabaja la iteración. La ficha dice qué abarca, por qué se eligió, qué queda por decidir sobre él y contra qué elementos del análisis se puede rastrear.

EL-01 — El sistema Bastion

Elemento	Descripción
Elemento	El sistema Bastion completo, tomado como una sola pieza todavía sin partes.
Qué abarca	Todo lo que queda dentro de la frontera del sistema: lo que el jugador usa en su equipo, lo que sostiene la partida mientras dura y lo que conserva cuentas, partidas y resultados. Fuera de la frontera quedan el proveedor del segundo factor (DEP-02), el correo (DEP-01) y la infraestructura que aloja los servidores y la base de datos (DEP-03, DEP-04).
Por qué se elige	Porque es el único elemento que existe. Bastion se construye desde cero y esta es la primera iteración: no hay elementos de iteraciones anteriores que refinar, así que el método indica empezar por el sistema mismo. Y porque es el recorte que corresponde al objetivo: decidir de qué extremo depende que una partida sea correcta no se puede hacer dentro de una parte del sistema, ya que la respuesta es precisamente qué partes hay y qué le toca a cada una.
Qué falta decidir sobre él	En cuántas partes se divide y cuál es la frontera de cada una.Cuál de esas partes produce el juego y cuál lo presenta. Dónde vive lo que la partida tiene en común mientras dura. Y qué se comprueba antes de que un mensaje pueda cambiar algo. Son decisiones que, tomadas de otra manera, dan sistemas distintos, y son las más caras de revertir (CRN-10).
Evidencia en los casos de uso	CU-21 reconstruye el tablero a partir de las Jugada registradas (paso 12) y confirma una transacción antes de notificar (pasos 15 a 19), sin decir de dónde sale el tablero con el que valida ni quién lo tiene. CU-21 resalta los surcos válidos (paso 3) y muestra el efecto del muro sobre el camino más corto (paso 5), sin decir qué parte hace ese cálculo. CU-20, CU-21 y CU-25 exigen en PRE-04 conexión disponible con la base de datos. Esos huecos son exactamente lo que esta iteración tiene que cerrar.
Trazabilidad	DRV-01, DRV-02, con DRV-03 como restricción. ASR-08, ASR-02, con ASR-01 como límite. QAS-05, QAS-01, QAS-07. CU-17 a CU-25. REQ-01, CON-02, CON-03, CON-04. DEC-01, DEC-02, DEC-03. CRN-10, CRN-23, CRN-12, CRN-02. STK-05, STK-15, STK-01.

4.5 Paso 4. Conceptos de diseño

Estos son los conceptos con los que se descompone el sistema. Cada ficha dice qué atiende y por qué se eligió frente a las alternativas, qué efectos trae consigo y contra qué elementos del análisis se puede rastrear. Las alternativas que se descartaron quedan registradas con su comparación completa en las fichas de decisión de la subsección 4.6.

CD-01 — Cliente-servidor

Elemento	Descripción
Estilo	Estilo arquitectónico.
Descripción	El sistema se parte en dos tipos de elemento: un cliente por jugador, que presenta y recoge lo que este hace, y un servidor común, que atiende a todos. Atiende DRV-01 porque es lo que permite que jugadores en equipos distintos vean el mismo juego: hay un lugar donde lo común existe una sola vez. Se elige frente al juego entre pares, donde cada equipo habla con el otro sin nadie en medio, que ahorra un salto de red pero deja el juego en manos de los participantes y no ofrece dónde guardar cuentas ni resultados.
Efectos	Cada acción del turno cuesta al menos un viaje completo de ida y vuelta, que se descuenta del segundo de QAS-01. Aparece la necesidad de un protocolo, que con CON-03 hay que construir. El servidor se vuelve un punto del que todos dependen, con lo que su caída afecta a todas las partidas.
Trazabilidad	DRV-01, DRV-03. ASR-08. QAS-01. CU-17 a CU-25. REQ-01, CON-02. DEC-01. CRN-02. STK-03, STK-01, STK-05.

CD-02 — El servidor produce el juego; el cliente lo presenta

Elemento	Descripción
Estilo	Estilo: reparto de responsabilidades entre los extremos.
Descripción	Lo que el jugador envía es una propuesta; lo que cambia la partida es la decisión del servidor. El cliente no calcula reglas ni resultados, ni siquiera para mostrarlos. Atiende DRV-02 en su raíz: si el cliente no produce nada, no hay nada suyo de lo que desconfiar, y la medida de QAS-05 deja de depender de qué tan bien se detecta un cliente alterado. Se elige frente a repartir el cálculo entre los dos extremos y comprobarlo después, que es lo habitual en juegos en línea y lo que CRN-12 señala como fuente de discrepancias visibles para el jugador.
Efectos	Descarta la validación optimista: no hay respuesta local que adelantar, así que todo el tiempo percibido es tiempo real y el margen de QAS-01 se estrecha. Obliga a que la notificación de turno lleve lo que la interfaz necesita para dibujar, que es de donde sale CD-08. A cambio, la corrección de la partida deja de depender de la versión del cliente.
Trazabilidad	DRV-02. ASR-02, ASR-01. QAS-05, QAS-01. CU-20 (pasos 1 a 3), CU-21 (pasos 3 a 7). DEC-02. CRN-23, CRN-12, CRN-10. STK-15, STK-05, STK-09.

CD-03 — Servicios separados dentro del servidor

Elemento	Descripción
Estilo	Estilo: descomposición en servicios.
Descripción	El servidor se divide por responsabilidad en cuatro servicios: comunicación, partidas, cuentas y acceso a datos, todos dentro de una misma pieza desplegable. Atiende DRV-02 al darle a la partida una frontera propia, separada de lo que la transporta y de lo que la guarda, y atiende DRV-01 al permitir que cada servicio se construya y se pruebe por separado. Se elige frente a un servidor de una sola pieza, que obligaría a decidirlo todo a la vez, y frente a servicios desplegados por separado, que con dos personas y 14 semanas añaden despliegue, versiones y saltos de red que QAS-01 no puede pagar.
Efectos	Fija las fronteras de las iteraciones siguientes: la comunicación es de DRV-03 y DRV-04, y el acceso a datos es de ASR-05. Obliga a definir interfaces internas antes de construir. Deja abierta la puerta de Q-02, porque la pieza se puede desplegar aparte sin cambiar su interior.
Trazabilidad	DRV-01, DRV-02, DRV-03. ASR-08, ASR-15. QAS-01. CON-13, CON-14, CON-16. DEC-01. CRN-07, CRN-11. Q-02. STK-05, STK-14, STK-03.

CD-04 — Las reglas del juego, en un componente aparte

Elemento	Descripción
Estilo	Patrón: modelo de dominio aislado.
Descripción	Un solo componente sabe qué jugadas son legales, qué muros se pueden colocar (RN-04), cuándo un jugador queda sin camino (RN-05) y cuándo termina la partida. No conoce pantallas ni conexiones: recibe una posición y una jugada, y responde. Atiende DRV-02 porque concentra en un solo lugar lo que decide si una acción es legítima, de modo que no hay dos versiones de la verdad que puedan discrepar (CRN-12). Se elige frente a dejar las reglas repartidas dentro del servicio de partidas, que las mezclaría con los turnos, los relojes y las notificaciones.
Efectos	Permite ejecutar partidas completas sin levantar red ni interfaz, que es la condición que ASR-06 necesitará después y lo que hace posible probar desde el primer incremento (CON-14). Obliga a que el servicio de partidas le pase la posición completa en cada consulta. Su costo de cálculo, sobre todo el de RN-05, queda dentro del turno y entra en el presupuesto de QAS-01.
Trazabilidad	DRV-02. ASR-02, ASR-06, ASR-01. QAS-05, QAS-12, QAS-01. CU-20, CU-21 (RN-04, RN-05). CON-14, CON-16. CRN-12, CRN-13. STK-06, STK-10, STK-05.

CD-05 — Copia de trabajo en memoria por partida, con registro persistente de cada jugada

Elemento	Descripción
Estilo	Patrón de gestión de estado.
Descripción	Mientras la partida dura, el servicio de partidas la tiene completa en memoria y decide contra esa copia; cada jugada aceptada se registra además en la base de datos, de la que la partida se puede reconstruir. Atiende DRV-02 en la parte que QAS-05 mide como “estado idéntico antes y después”: solo se puede afirmar de algo que tenga un dueño único y conocido. Se elige frente a leer la partida completa del almacenamiento en cada jugada, que metería la latencia de DEP-03 dentro de cada turno.
Efectos	Cumple DEC-03 sin poner el almacenamiento en el camino de la respuesta. Obliga a reconstruir desde las jugadas registradas al recuperar, que es lo que CU-21 (paso 12) y CU-26 (paso 9) ya describen. Abre una diferencia temporal entre lo respondido y lo escrito, que ASR-05 acotará en cinco segundos. Y hace que la memoria del servidor crezca con el número de partidas simultáneas.
Trazabilidad	DRV-01, DRV-02. ASR-02, ASR-05, ASR-01. QAS-05, QAS-09, QAS-01. CU-20, CU-21 (paso 12), CU-25. CON-04, DEP-03, DEP-04. DEC-03. TEN-06. CRN-19, CRN-10. STK-13, STK-14.

CD-06 — Una partida atiende sus mensajes de uno en uno

Elemento	Descripción
Estilo	Patrón de concurrencia: objeto activo.
Descripción	Cada partida tiene su propia cola y procesa lo que le llega en orden, uno a la vez, mientras las demás partidas avanzan en paralelo. Atiende DRV-02 en la parte que exige que el estado quede idéntico tras cada mensaje rechazado: sin este orden, dos mensajes simultáneos podrían dejar la partida en una posición que ninguna jugada legal produce. Además, una jugada fuera de orden se detecta de forma natural (FA-07 del CU-21). Se elige frente a un bloqueo compartido por todo el servidor, que serializaría entre sí las 20 partidas de QAS-05 y rompería QAS-01.
Efectos	Cada partida se puede medir por separado, lo que facilita comprobar QAS-01. El trabajo por jugada tiene que ser corto, porque mientras dura bloquea a su propia cola: eso obliga a que las escrituras lentas salgan del camino. Nada fuera de la partida puede tocar su estado.
Trazabilidad	DRV-02. ASR-02, ASR-01. QAS-05, QAS-01. CU-20, CU-21 (FA-07, NOT_YOUR_TURN). CRN-02, CRN-10. STK-01, STK-15, STK-05.

CD-07 — Cada mensaje se valida, se autentica y se autoriza antes de tocar la partida

Elemento	Descripción
Estilo	Tácticas de seguridad: validar la entrada, autenticar y autorizar a los actores.
Descripción	Todo mensaje pasa por el mismo filtro antes de llegar al servicio de partidas: se comprueba que esté bien formado y que su versión sea la esperada, que corresponda a una sesión abierta y que esa cuenta participe en esa partida, como hace CU-21 en su paso 8 y resuelve en FA-05 con SESSION_INVALID y NOT_A_PARTICIPANT. Atiende DRV-02 en dos de los tres frentes de QAS-05: los 50 mensajes mal formados o de otra versión, y las solicitudes de datos de la cuenta del rival. Se elige frente a confiar en el identificador de jugador que el propio mensaje trae, que es justo lo que un cliente alterado falsifica, y frente a que cada servicio se defienda por su cuenta, que repartiría la misma comprobación por todo el servidor.
Efectos	El servicio de partidas solo recibe mensajes bien formados y de quien dice ser, y puede ocuparse de reglas y turnos. Obliga a que el servicio de cuentas exponga la validez de una sesión al de partidas. Agrega un paso por mensaje, con su costo dentro de QAS-01, y deja preparado el sitio donde DRV-04 pondrá la protección de lo que viaja.

Elemento	Descripción
Trazabilidad	DRV-02, DRV-03, DRV-04. ASR-02, ASR-03. QAS-05, QAS-06. CU-20, CU-21 (paso 8, FA-05, EX-02, EX-03), CU-01. CON-03. CRN-23, CRN-11. STK-15, STK-10, STK-02.

CD-08 — Cada destinatario recibe solo lo que puede conocer

Elemento	Descripción
Estilo	Táctica de seguridad: limitar el acceso a la información, con origen en CD-02.
Descripción	El servidor arma lo que envía en función de quién lo va a recibir: al jugador en turno le manda además las casillas y los surcos donde puede jugar y el efecto de cada muro sobre el camino más corto; a nadie le manda datos de la cuenta de otro. Atiende el tercer frente de QAS-05, las 0 respuestas con datos privados del rival, y hace posible CD-02: la interfaz resalta y previsualiza con lo que ya recibió, sin calcular nada. Se elige frente a enviar a todos el mismo mensaje completo y dejar que cada cliente muestre lo que le toca, que confía la confidencialidad al equipo del jugador, y frente a preguntar al servidor cada vez que el jugador arrastra un muro por el tablero, que multiplicaría los viajes dentro del turno.
Efectos	El mensaje de turno crece, con su costo para el protocolo de DRV-03 y para el presupuesto de QAS-01. Mantiene CU-20 y CU-21 tal como están escritos de cara al jugador. Obliga a que el componente de reglas sepa enumerar las jugadas legales, y no solo juzgar una. Deja resuelto de antemano el caso del espectador de ASR-09, que recibirá su propia versión.
Trazabilidad	DRV-02. ASR-02, ASR-09, ASR-01. QAS-05, QAS-01, QAS-02. CU-20 (pasos 1 a 3), CU-21 (pasos 3 a 5, FA-02), CU-34. DEC-02. REQ-07. CRN-01, CRN-02, CRN-23. STK-01, STK-09, STK-15.

CD-09 — Cada intento rechazado queda registrado con su cuenta y su hora

Elemento	Descripción
Estilo	Táctica de seguridad: detectar y auditar.
Descripción	Todo mensaje que el filtro o la partida rechazan se anota con la cuenta que lo envió y el momento en que llegó. Atiende la última parte de la medida de QAS-05, el 100 % de los intentos registrados, y es lo que convierte un rechazo silencioso en evidencia. Se elige frente a contar los rechazos sin identificarlos, que no permitiría investigar ni sostener una sanción.
Efectos	Exige que todo mensaje llegue ligado a una cuenta, lo que depende de CD-07. Genera escritura adicional que no puede entrar en el camino de la respuesta, por el mismo motivo que la de CD-05. Da la base sobre la que después se apoyan la moderación de CU-42 a CU-45 y las bitácoras de CU-48.
Trazabilidad	DRV-02. ASR-02, ASR-11. QAS-05, QAS-07. QA-08. CU-42 a CU-45, CU-48. TEN-06. CRN-18, CRN-23. STK-15, STK-16, STK-17.

Elementos que resultan de la descomposición

Al aplicar los nueve conceptos al sistema, EL-01 deja de ser una sola pieza. Estos son los elementos que quedan, y a ellos se refieren las fichas de decisión.

ID	Elemento	Responsabilidad	Conceptos
C-01	Cliente de escritorio	Presenta el tablero, recoge lo que el jugador hace y muestra lo que el servidor le envía. No calcula reglas ni resultados.	CD-01, CD-02, CD-08
C-02	Punto de entrada de comunicación	Recibe y envía mensajes, comprueba su forma y su versión, y resuelve a qué sesión y a qué cuenta corresponde cada uno.	CD-01, CD-03, CD-07
C-03	Servicio de partidas	Sostiene cada partida en curso, controla turnos y relojes, acepta o rechaza las acciones y notifica el resultado.	CD-03, CD-05, CD-06, CD-08

ID	Elemento	Responsabilidad	Conceptos
C-04	Componente de reglas	Dice qué jugadas son legales sobre una posición dada y cuándo termina la partida. No conoce pantallas ni conexiones.	CD-04
C-05	Servicio de cuentas y sesiones	Abre y cierra sesiones, y responde si una sesión sigue siendo válida y a qué cuenta pertenece.	CD-03, CD-07
C-06	Servicio de acceso a datos	Es el único que habla con la base de datos; registra jugadas y resultados y devuelve lo necesario para reconstruir una partida.	CD-03, CD-05
C-07	Base de datos	Conserva cuentas, partidas, participaciones y jugadas. Es la copia de referencia.	CD-05
C-08	Bitácora de seguridad	Conserva los intentos rechazados con su cuenta y su hora.	CD-09

Riesgos que dejan abiertos estos conceptos

ID	Riesgo	Afecta a
R-01	El viaje completo de ida y vuelta más el arbitraje no caben en el segundo de QAS-01 cuando se les suma la protección de lo que viaja, que todavía no se ha decidido.	ASR-01, ASR-03
R-02	Sin ninguna respuesta local, la interfaz se percibe lenta aunque el tiempo medido cumpla, sobre todo al arrastrar un muro por el tablero.	ASR-01; STK-09, STK-01
R-03	El cálculo de camino de RN-05 dentro del turno domina el tiempo de respuesta en el tablero avanzado, que es justo el caso de QAS-01.	ASR-01, ASR-06
R-04	La copia en memoria y lo registrado divergen si el proceso se detiene entre una y otra, y la partida se reconstruye distinta.	ASR-05
R-05	Todas las partidas en curso viven en un mismo proceso: si se reinicia, se pierden todas a la vez hasta donde alcance lo registrado.	ASR-04, ASR-05; DEP-04
R-06	Una sesión válida capturada permite suplantar al jugador mientras no exista la protección del tránsito, que es de DRV-04.	ASR-03, ASR-02
R-07	Registrar cada intento rechazado permite que un atacante llene la bitácora a propósito y degrade el servicio.	ASR-02; QAS-05
R-08	Construir el protocolo propio, los cuatro servicios y el componente de reglas excede lo que dos personas pueden terminar en 14 semanas.	ASR-15; CRN-07

4.6 Fichas de decisión

Cada ficha registra una decisión de esta iteración con las alternativas que se compararon, los criterios con que se compararon y lo que la decisión cuesta. Las tres primeras corresponden a las decisiones que el equipo traía tomadas (DEC-01 a DEC-03) y que aquí quedan justificadas; las dos últimas son nuevas.

ADR-001 · El servidor produce el juego y el cliente solo lo presenta

Campo	Contenido
Estado	Aceptado.
Contexto	Hay que decidir de qué extremo depende que una partida sea correcta. Es la decisión que CRN-10 señala como la más cara de cambiar y de la que cuelga el significado de cada mensaje. En juego: ASR-02, que exige que solo las acciones legítimas cambien la partida aunque el equipo del jugador esté alterado (CRN-23), y ASR-08, que exige que todos los participantes vean el mismo juego. Como límite, ASR-01.
Alternativas	A. El cliente calcula la jugada y el servidor la retransmite. A favor: la respuesta es inmediata, porque no hay que esperar a nadie, y el servidor hace muy poco trabajo. En contra: el juego queda en manos de un equipo que se supone hostil, y QAS-05 sería imposible de cumplir. B. El cliente calcula y el servidor comprueba después, revirtiendo si no coincide. A favor: conserva la respuesta inmediata y detecta el fraude. En contra: el jugador ve jugadas aceptadas que luego se revierten, que es justo lo que CRN-12 teme, y obliga a mantener las reglas en los dos extremos. C. El cliente propone y el servidor decide; el cliente no calcula nada. A favor: no hay nada que el cliente produzca de lo que haya que desconfiar, y la corrección deja de depender de su versión. En contra: cada acción cuesta un viaje completo y todo el tiempo percibido es real.
Criterios	Que QAS-05 sea alcanzable; que el jugador nunca vea revertirse algo que se le mostró como aceptado (CRN-12); que quepa en el segundo de QAS-01; y que dos personas puedan construirlo en 14 semanas (ASR-15).
Decisión	C, que es lo que DEC-02 ya proponía (conceptos CD-01 y CD-02; tácticas: limitar el acceso a la información y validar la entrada). A se descarta porque hace imposible la medida del objetivo; B, porque duplica las reglas, choca con CRN-12 y no ahorra trabajo al servidor, que igual tiene que calcularlo todo para comprobar.
Consecuencias	
Qué se ganó	Una sola versión de la verdad: la partida es lo que el servidor dice que es, y un cliente modificado no cambia eso. El objetivo se vuelve comprobable, porque basta medir sobre el servidor. La corrección del juego deja de depender de qué versión del cliente tenga cada jugador, lo que simplifica las actualizaciones. Y todo lo que hay que proteger queda de un solo lado.
Qué se pagó	Cada acción del turno cuesta un viaje completo de ida y vuelta, sin nada local que adelantar: el presupuesto de QAS-01 se consume desde el primer momento y no queda margen para respuestas optimistas. El servidor carga con todo el cálculo, incluido el de RN-05, que es el más caro. Y la interfaz depende de que el servidor le mande lo que necesita para dibujar, lo que obliga a CD-08 y engorda los mensajes.
Riesgos	R-01, R-02, R-03.
Elementos afectados	C-01, C-02, C-03, C-04.

ADR-002 · El servidor se divide en cuatro servicios dentro de una sola pieza desplegable

Campo	Contenido
Estado	Aceptado.
Contexto	Aceptada ADR-001, hay que decidir cómo se organiza por dentro el servidor. En juego: ASR-02, que necesita que la partida tenga una frontera propia para poder afirmar que su estado no cambió; ASR-15, que fija dos personas y 14 semanas con un incremento verificable cada semana; y ASR-01, porque cada frontera que un mensaje cruza cuesta tiempo. DRV-03 obliga además a que el transporte sea un componente propio.
Alternativas	A. Un servidor de una sola pieza, sin fronteras internas. A favor: es lo más rápido de arrancar y no cuesta ningún salto. En contra: comunicación, partida y datos quedan mezclados, así que habría que decidirlos a la vez y no se podría probar nada por separado, en contra de CON-14. B. Un servicio por área desplegado por separado, cada uno con su proceso. A favor: se pueden escalar y desplegar por separado, y las fronteras quedan garantizadas. En contra: añade despliegue, versionado y saltos de red a un equipo de dos personas (CRN-07), y cada salto se descuenta de QAS-01. C. Cuatro servicios con responsabilidades separadas dentro de una misma pieza desplegable: comunicación, partidas, cuentas y acceso a datos. A favor: fronteras claras y llamadas dentro del mismo proceso, y cada servicio se construye y se prueba por separado. En contra: la separación depende de la disciplina del equipo, porque nada impide saltársela.
Criterios	Que la partida tenga un dueño con frontera propia (ASR-02); que se pueda entregar un incremento verificable por semana (ASR-15, CON-14); que no se agregue latencia evitable (ASR-01); y que quepa en el esfuerzo de dos personas (CRN-07).
Decisión	C, coherente con DEC-01 (concepto CD-03). A se descarta por CON-14 y porque haría inseparables decisiones que pertenecen a iteraciones distintas; B, por su costo de operación y de latencia, desproporcionado para la escala del proyecto. Si Q-02 se resuelve a favor, la misma pieza se despliega aparte sin cambiar su interior.
Consecuencias	
Qué se ganó	Las iteraciones siguientes tienen dónde caer sin pisarse: el transporte es de DRV-03 y DRV-04, el acceso a datos es de ASR-05, y ninguna de esas decisiones obliga a abrir el servicio de partidas. Cada servicio se puede probar solo, lo que hace posible el incremento semanal. Y las llamadas entre servicios no cuestan red.
Qué se pagó	Hay que definir las interfaces internas antes de construir, que es trabajo de diseño por adelantado. La separación no está garantizada por nada más que por el acuerdo del equipo. Y al ser una sola pieza, todas las partidas comparten suerte: un reinicio las afecta a todas.
Riesgos	R-05, R-08.
Elementos afectados	C-02, C-03, C-05, C-06.

ADR-003 · Las reglas del juego viven en un componente que no conoce pantallas ni conexiones

Campo	Contenido
Estado	Aceptado.
Contexto	Aceptada ADR-001, el servidor tiene que calcularlo todo, y hay que decidir dónde vive ese cálculo. En juego: ASR-02, porque las reglas son lo que distingue una acción legítima de una que no lo es; ASR-06, que más adelante exigirá cambiar los valores de las reglas sin tocar código y verificar una partida completa sin intervención manual; y ASR-01, porque el cálculo de RN-05 ocurre dentro del turno.
Alternativas	A. Las reglas repartidas dentro del servicio de partidas, junto a turnos, relojes y notificaciones. A favor: no hay que definir ninguna interfaz nueva. En contra: no se puede ejecutar una partida sin levantar el servidor entero, y cualquier cambio de reglas obliga a tocar el servicio que arbitra. B. Las reglas duplicadas en cliente y servidor, para que el cliente pueda previsualizar. A favor: la interfaz respondería al instante. En contra: contradice ADR-001 y crea dos versiones que tarde o temprano difieren (CRN-12). C. Un componente único de reglas, consumido solo por el servicio de partidas, que recibe una posición y una jugada y responde. A favor: se ejecuta sin red ni interfaz, se puede probar con partidas automáticas y concentra en un solo lugar lo que define la legalidad. En contra: obliga a pasarle la posición completa en cada consulta y a mantener una interfaz más.
Criterios	Que exista una sola definición de lo que es legal (ASR-02, CRN-12); que se pueda verificar sin intervención manual desde el primer incremento (ASR-06, CON-14); y que su costo quepa en el turno (ASR-01).
Decisión	C (concepto CD-04; patrón: modelo de dominio aislado). A se descarta porque haría imposible probar sin levantarlo todo y encarecería cualquier cambio de reglas; B, porque contradice la decisión anterior.
Consecuencias	
Qué se ganó	Se pueden ejecutar partidas completas en pruebas automáticas desde la primera semana, sin red ni interfaz, que es lo que sostiene el incremento semanal y lo que ASR-06 necesitará después. Hay una sola definición de lo legal, así que los dos extremos no pueden discrepar. Y las reglas se pueden cambiar sin tocar el arbitraje.
Qué se pagó	Una interfaz interna más que definir y mantener, y el costo de pasarle la posición completa en cada consulta. El cálculo de RN-05 queda dentro del turno y compite por el presupuesto de QAS-01, sin que todavía se sepa cuánto consume.
Riesgos	R-03.
Elementos afectados	C-03, C-04.

ADR-004 · La partida en curso vive en memoria y cada jugada aceptada se registra

Campo	Contenido
Estado	Aceptado.
Contexto	Hay que decidir dónde está la partida mientras se juega. QAS-05 exige poder afirmar que su estado es idéntico antes y después de cada mensaje rechazado, lo que supone un dueño único. DEC-03 exige que la partida no se juegue sin almacenamiento disponible, y CU-20, CU-21 y CU-25 lo repiten en PRE-04. ASR-01 impide meter en el turno la latencia de una infraestructura que no controlamos (DEP-03, DEP-04). ASR-05 acotará después lo que se puede perder.
Alternativas	A. Sin estado en el servidor: cada jugada lee la partida completa de la base de datos, valida y escribe. A favor: no hay nada que perder en un reinicio y la base de datos es siempre la verdad. En contra: mete dos accesos a un servicio externo dentro de cada turno, lo que hace muy difícil QAS-01, y es lo que CU-21 describe hoy en sus pasos 12 a 19. B. Solo memoria: la partida se escribe al terminar. A favor: es lo más rápido posible. En contra: un reinicio pierde partidas enteras y contradice DEC-03 y ASR-05. C. Copia de trabajo en memoria por partida, atendida de una en una, con cada jugada aceptada registrada en la base de datos. A favor: se decide y se responde sin esperar al almacenamiento, y la partida se puede reconstruir desde lo registrado. En contra: hay una ventana entre lo respondido y lo escrito, y la memoria crece con las partidas simultáneas.
Criterios	Que la partida tenga un dueño único y conocido (ASR-02, QAS-05); que el almacenamiento quede fuera del camino de la respuesta (ASR-01); que se pueda reconstruir lo perdido (ASR-05); y que se respete DEC-03.
Decisión	C (conceptos CD-05 y CD-06; patrones: copia de trabajo con respaldo y objeto activo por partida). A se descarta por QAS-01; B, porque contradice DEC-03 y haría imposible ASR-05. La ventana entre responder y escribir queda abierta a propósito: cuánto puede durar lo decide la iteración de ASR-05.
Consecuencias	
Qué se ganó	Se puede afirmar que el estado de la partida no cambió, porque hay un solo lugar donde vive. El turno no paga la latencia de una infraestructura externa. La partida se puede reconstruir desde las jugadas registradas, que es lo que CU-21 y CU-26 ya suponían. Y, al atender los mensajes de uno en uno, dos jugadas simultáneas no pueden dejar la partida en una posición imposible.
Qué se pagó	Existe una ventana en la que lo respondido todavía no está escrito, con el riesgo de que las dos versiones difieran. La memoria del servidor crece con el número de partidas en curso. Y el trabajo dentro del turno tiene que ser corto, porque bloquea la cola de su propia partida, lo que obliga a sacar de ahí toda escritura lenta.
Riesgos	R-04, R-05, R-01.
Elementos afectados	C-03, C-06, C-07.

ADR-005 · Todo mensaje se comprueba en el punto de entrada y lo rechazado queda registrado

Campo	Contenido
Estado	Aceptado.
Contexto	QAS-05 mide tres cosas a la vez: que se rechacen 200 mensajes inválidos, entre ellos 50 mal formados o de otra versión; que ninguna respuesta lleve datos privados del rival; y que el 100 % de los intentos quede registrado con cuenta y hora. Hay que decidir dónde se hacen esas comprobaciones. En juego: ASR-02 y, de rebote, ASR-03, porque el mismo punto tendrá que albergar después la protección de lo que viaja (DRV-04). CON-03 impide heredar ninguna de estas comprobaciones del transporte.
Alternativas	A. Confiar en el identificador de jugador que trae el mensaje y comprobar solo la legalidad de la jugada. A favor: es lo más barato. En contra: un cliente alterado falsifica ese identificador, así que no cubre ni la suplantación ni las solicitudes de datos ajenos. B. Que cada servicio se defienda por su cuenta. A favor: cada uno comprueba exactamente lo que necesita. En contra: la misma comprobación queda repetida en cuatro lugares que tarde o temprano difieren, y basta olvidarla en uno para abrir el hueco. C. Un punto de entrada único que comprueba forma y versión, resuelve la sesión y la cuenta, autoriza contra la participación en esa partida y registra cada rechazo; el servicio de partidas solo recibe lo que ya pasó. A favor: una sola definición de lo que se acepta, y un solo sitio donde añadir después la protección del tránsito. En contra: agrega un paso por mensaje y concentra trabajo en un punto que puede saturarse.
Criterios	Cubrir los tres frentes que QAS-05 mide; no confiar en nada que el cliente envíe sobre sí mismo (CRN-23); dejar las reglas de acceso en un solo lugar; y que el costo por mensaje quepa en QAS-01.
Decisión	C (conceptos CD-07, CD-08 y CD-09; tácticas: validar la entrada, autenticar y autorizar a los actores, limitar el acceso a la información y auditar). A se descarta porque no resiste el supuesto de CRN-23; B, porque reparte la misma regla en cuatro sitios, y una regla repetida es una regla que acabará siendo distinta.
Consecuencias	
Qué se ganó	Las reglas de acceso están en un solo lugar, y ahí mismo se podrá poner después la protección del tránsito sin tocar los demás servicios. El servicio de partidas recibe solo mensajes bien formados y de quien dice ser, así que se ocupa de reglas y turnos y nada más. Cada rechazo queda como evidencia, que es lo que QAS-07 pide y lo que la moderación de CU-42 a CU-45 necesitará.
Qué se pagó	Un paso más por mensaje, con su costo dentro del segundo de QAS-01. El punto de entrada concentra trabajo y se vuelve un lugar sensible a la saturación. Y el registro de rechazos es escritura adicional que hay que mantener fuera del camino de la respuesta y acotar, porque un atacante puede provocarla a voluntad.
Riesgos	R-06, R-07, R-01.
Elementos afectados	C-02, C-03, C-05, C-08.

Qué queda para la siguiente iteración

Las cinco decisiones cubren los dos drivers de esta iteración y dejan el sistema partido en ocho elementos. Lo que sigue es DRV-03 sobre C-02: cómo se delimita un mensaje, cómo se versiona y cómo se abre una sesión sobre sockets TCP sin WebSockets. De los riesgos abiertos, R-01 y R-03 se resolverán al repartir el presupuesto de ASR-01; R-04 y R-05, al decidir qué se escribe y cuándo (ASR-05); y R-06, con la protección del tránsito (DRV-04).